

SmartCare v0.4 - Domain Implementation Workbook

Week 7 student resource

1. UML-to-Code Trace

UML element	Python element	Implemented?	Notes
Class: Appointment	class Appointment:	YES	Core domain class representing individual bookings.
Class: ClinicController	class ClinicController:	YES	'Root' class orchestrating system-level workflows.
Class: Patient	class Patient:	YES	Core entity holding patient details and managing appointment_history.
Class: Practitioner	class Practitioner:	YES	Core entity holding specialty/room details and filtering daily schedules.
Enumeration: AppointmentStatus	class AppointmentStatus(Enum):	NOT YET	Inherits from enum.Enum to enforce type-safe status values (SCHEDULED, ARRIVED, IN_PROGRESS, COMPLETED, CANCELLED).
Private Attribute: - appointment_id : String	self._appointment_id + @property	NOT YET	Private attribute initialized in init with whitespace stripping; exposed via read-only property.
Private Attribute: - patient_id : String	self._patient_id + @property	NOT YET	Private attribute initialized in init ; exposed via read-only property.
Private Attribute: - practitioner_id : String	self._practitioner_id + @property	NOT YET	Private attribute initialized in init ; exposed via read-only property.
Private Attribute: - date_time : String	self._date_time + @property	NOT YET	Private attribute

			initialized in init ; exposed via read-only property.
status : AppointmentStatus	self._status + @property	NOT YET	Private attribute managing internal state machine value; exposed via read-only property.

2. Domain Invariants

Class	Invariant / rule	How protected
Patient	Immutable Identity: The patient_id must never change or be modified after instantiation.	Stored as a private attribute (_patient_id) with no setter method, exposed only via a read-only @property.
Patient	History Integrity: The appointment_history must only contain valid, assigned Appointment objects and cannot be cleared or overwritten externally.	Encapsulated via add_to_history(); queries via get_appointment_history() return a read-only copy or list reference rather than direct attribute access.
Practitioner	Schedule Integrity: A practitioner cannot have two active appointments (SCHEDULED, ARRIVED, IN_PROGRESS) at the exact same date and time slot.	Checked prior to booking creation via Appointment.check_collision() or ClinicController.book_appointment().
Appointment	Valid State Transitions: Status modifications must strictly follow allowed forward paths (SCHEDULED > ARRIVED > IN_PROGRESS > COMPLETED / CANCELLED).	Enforced in update_status() by evaluating new_status against the ALLOWED_TRANSITIONS state machine dictionary.
Appointment	Mandatory Cancellation Reason: An appointment cannot transition to or be instantiated as CANCELLED without an explicit, non-empty text reason.	Enforced in both init validation and cancel_appointment(), raising an AppointmentException if reason.strip() is empty.
ClinicController	Referential Integrity: An appointment cannot be booked unless both the referenced patient_id and practitioner_id exist in the registered registries.	Enforced in book_appointment() by verifying key presence in _patients and _practitioners dictionaries before instantiating the booking.

3. Composition / Inheritance Decisions

Relationship	Decision	Rationale
Appointment > Patient	Association	An Appointment is not a type of Patient (failing the "IS-A" inheritance check). Both entities have independent lifespans;

		deleting or cancelling an appointment does not delete the patient record.
Appointment > Practitioner	Association	A Practitioner exists independently of individual bookings. An appointment references a practitioner to define who conducts the consultation, but removing an appointment leaves the practitioner intact in the system.
Doctor > Practitioner	Inheritance	A Doctor is a specialized type of Practitioner by inheriting general practitioner attributes (id, name, schedule) while adding specialized medical fields or capabilities.
Appointment > AppointmentStatus	Dependency	AppointmentStatus is an enumeration used as a constrained, type-safe attribute within Appointment. It represents a value domain rather than a structural composition or class hierarchy.
ClinicController > Domain Entities (Patient, Practitioner, Appointment)	Aggregation	The ClinicController acts as an aggregate root container managing the dict collections. Aggregation is chosen because domain entity records represent real-world clinical data that persist logically even if system workflows or controller instances are re-initialized.

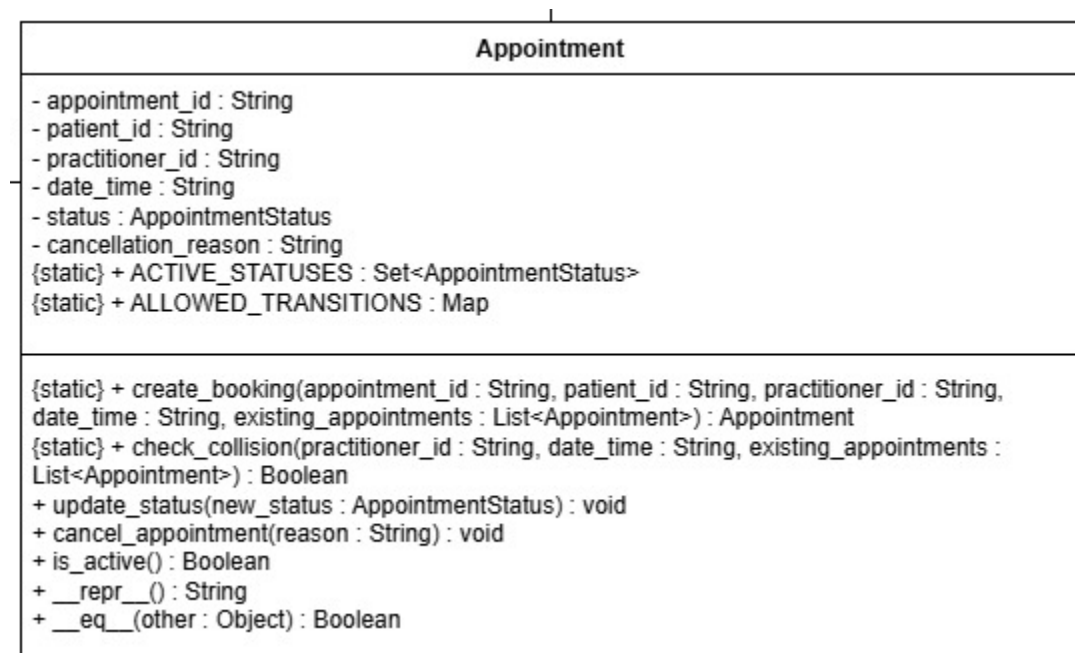
4. AI Pair-Programming Record

AI contribution	Conforms?	Decision	Reason	Verification
ClinicController Class: Recommended adding a controller class to manage Patient, Practitioner, and Appointment collections and handle foreign key presence validation	YES	ACCEPTED	SEPARATES high-level workflow orchestration and registry lookups from core entity business logic.	Verified via CRC card responsibilities and UML class diagram aggregate root relationships.
AppointmentStatus Enumeration: Modeled status as an Enum class rather than a plain string primitive.	YES	ACCEPTED	Prevents invalid states, typos, and improper string assignments, providing type safety for state machine transitions (ALLOWED_TRANSITIONS).	Checked Python Enum implementation and state transition logic in update_status().

Cancellation Invariant in init : Added a validation check in init to raise an AppointmentException if instantiated as CANCELLED without a cancellation_reason.	YES	ACCEPTED	Ensures class invariants hold regardless of whether an object is initialized directly or updated via cancel_appointment().	Tested init edge cases with empty strings and whitespace-only cancellation reasons.

5. Updated UML

Insert updated UML only if implementation revealed a justified design change. Explain every change.



New Method Signatures: Added `__repr__()` and `__eq__(other: Object) : Boolean` to the operations compartment for structural equality checking.